Authorization in SQL

Authorization in SQL

Definition

Authorization in SQL specifies which users (or roles) are allowed to perform which operations on which parts of the database. It controls both **data access** and **schema modifications**.

1. Forms of Authorization on Data

Data Access Privileges

- **READ (SELECT)** – Allows reading but not modifying data.
- **INSERT** – Allows adding new data but not modifying existing data.
- **UPDATE** – Allows modification but not deletion of data.
- **DELETE** – Allows deletion of data.

2. Forms of Authorization on Schema

Schema Modification Privileges

- **INDEX** – Allows creation and deletion of indexes.
- **RESOURCES** – Allows creation of new relations.
- **ALTERATION** – Allows adding or deleting attributes in a relation.
- **DROP** – Allows deletion of relations.

3. Granting of Privileges

Syntax

```
GRANT <privilege list>
ON <relation or view>
TO <user_or_role_list>
[WITH GRANT OPTION];
```

Key Points

- `<user_or_role_list>` can be:
  - A specific **user-id**.
  - **PUBLIC** (all valid users).
  - A **role**.
- Privileges on a view do not imply privileges on the base tables.
- The grantor must already hold the privilege or be the database administrator.

Example

Grant `SELECT` privilege on `instructor` to three users:

```
GRANT SELECT ON instructor TO U1, U2, U3;
```

4. Privileges in SQL

Common Privileges

- **SELECT** – Read access to a relation or view.
- **INSERT** – Ability to add tuples.
- **UPDATE** – Modify tuples using UPDATE.
- **DELETE** – Remove tuples.
- **ALL PRIVILEGES** – Grants all allowable privileges.
- **REFERENCES** – Create foreign keys referencing a table.

5. Revoking Privileges

Syntax

```
REVOKE <privilege list>
ON <relation or view>
FROM <user_or_role_list>
[CASCADE | RESTRICT];
```

Key Rules

- `ALL` in privilege list revokes all privileges the user holds.
- If `PUBLIC` is used, all users lose the privilege except those granted it explicitly.
- If the same privilege is granted by multiple users, the revokee may still retain it after one revocation.
- Revocation cascades to dependent privileges if `CASCADE` is specified.

Example

```
REVOKE SELECT ON branch FROM U1, U2, U3;
```

6. Roles

Definition

A **role** is a named collection of privileges. Privileges can be granted to roles, and roles can be assigned to users or other roles.

Examples

```
CREATE ROLE instructor;
GRANT SELECT ON takes TO instructor;
GRANT instructor TO Amit;

CREATE ROLE teaching_assistant;
GRANT teaching_assistant TO instructor;

CREATE ROLE dean;
GRANT instructor TO dean;
GRANT dean TO Satoshi;
```

7. Authorization on Views

Example

```
CREATE VIEW geo_instructor AS
SELECT *
FROM instructor
WHERE dept_name = 'Geology';

GRANT SELECT ON geo_instructor TO geo_staff;
```

Important Notes

- A user can have privileges on a view without having privileges on the underlying tables.
- If the creator of the view lacks privileges on some base table columns, the view creation fails.

8. Transfer of Privileges

WITH GRANT OPTION

A privilege can be granted with the ability to pass it to others:

```
GRANT SELECT ON department TO Amit WITH GRANT OPTION;
```

Revoking with CASCADE or RESTRICT

```
REVOKE SELECT ON department FROM Amit, Satoshi CASCADE;
REVOKE SELECT ON department FROM Amit, Satoshi RESTRICT;
```

Authorization Graph

- Represented as a directed graph with users as nodes.
- Edge $U_i \rightarrow U_j$ means U_i granted a privilege to U_j.
- A user has a privilege if there is a path from the DBA (root) to the user.

9. Row-Level Authorization

Definition

Row-Level Security (RLS) restricts which rows a user can access based on a policy.

Example in PostgreSQL

```
ALTER TABLE Student ENABLE ROW LEVEL SECURITY;
CREATE POLICY student_policy
ON Student
FOR SELECT
USING (advisor_id = current_user_id());
```